neutron-diffuse

Manual

3D Reciprocal-Space Diffuse Neutron Scattering Analysis
Powder Ring Removal · Bragg Punch & Fill · 3D-ΔPDF Transform

Version 0.1.0
June 2026

Language	Python ≥ 3.10
License	MIT
Dependencies	numpy, scipy, h5py, matplotlib
Instrument	Mantid NeXus (MDHistoWorkspace)
Reference material	$TbTi_3Bi_4$ (22 K / 45 K / 100 K)

Tsung-Han Yang

Table of Contents

1. Introduction

neutron-diffuse is a Python 3.10+ toolkit for the complete analysis of three-dimensional (3D) diffuse neutron scattering volumes. It implements a production-ready four-stage pipeline that transforms raw single-crystal Mantid NeXus output into real-space 3D difference pair distribution functions (3D-ΔPDF):

Stage 1 — Powder-ring removal. Subtracts azimuthally smooth ring contamination from the cryostat, sample holder, and sample environment without masking real diffuse signal.

Stage 2 — Bragg punch. Identifies and masks sharp Bragg and satellite peaks using lattice-aware integer-node detection, data-driven shaping, and an hkl-agnostic search mode for off-integer satellites.

Stage 3 — Backfill. Replaces masked voxels with physically reasonable estimates derived from the surrounding radial background or crystal symmetry equivalents.

Stage 4 — 3D-ΔPDF transform. Converts the cleaned reciprocal-space volume to a real-space pair-correlation map via a correctly centred 3D discrete Fourier transform with apodization and smooth-background subtraction.

The package is designed around a single core data structure (HKLVolume) that carries the 3D intensity array, per-voxel standard deviations, a validity mask, HKL coordinate axes, and the crystal orientation (UB) matrix. All pipeline stages operate on this type, making it straightforward to inspect intermediate results, apply individual stages, or extend the pipeline with custom algorithms.

Visualization is provided through a thin, scriptable layer of Matplotlib primitives: 2D plane slices, radial profiles, azimuthal ring-texture maps, a 2×2 overview diagnostic, and three interactive orthogonal-cut real-space viewers for quality assurance at every stage.

1.1 Scope of This Manual

This manual covers the background physics and mathematics of 3D diffuse neutron scattering analysis, the detailed algorithms and their theoretical basis, a step-by-step workflow guide, the full Python API, configuration parameters, worked examples for the reference $TbTi_3Bi_4$ dataset at 22 K, 45 K, and 100 K, interactive visualization, known limitations, and a complete reference list.

1.2 What neutron-diffuse Does Not Do

The package does not perform raw detector reduction, normalization to an absolute cross-section, or Reverse Monte Carlo (RMC) structural refinement. It assumes the input is a Mantid background-subtracted MDHistoWorkspace volume (*_cc_sub_bkg.nxs). For the reduction steps that precede this, use Mantid directly.

2. Background Theory

2.1 Neutron Scattering Fundamentals

In a neutron scattering experiment a monochromatic beam of neutrons with wavevector $\mathbf{k}_i$ is scattered by the sample. Neutrons with final wavevector $\mathbf{k}_f$ are detected as a function of the momentum transfer

$$\mathbf{Q} = \mathbf{k}_f - \mathbf{k}_i \quad (|\mathbf{Q}| = Q = 4\pi \sin\theta / \lambda)$$

For an elastic measurement ($|\mathbf{k}_i| = |\mathbf{k}_f|$) on a single crystal, the scattered intensity $I(\mathbf{Q})$ is recorded over a 3D grid in reciprocal space indexed by the fractional Miller indices (h, k, l).

The total measured intensity separates into two components:

$$I_{total}(\mathbf{Q}) = I_{Bragg}(\mathbf{Q}) + I_{diffuse}(\mathbf{Q})$$

The **Bragg contribution** I_{Bragg} consists of sharp peaks at integer lattice nodes (and satellite peaks for modulated structures). It encodes the average periodic structure. The **diffuse contribution** $I_{diffuse}$ is the remaining broad, continuous signal that encodes deviations from the average: atomic displacements, occupancy disorder, short-range magnetic correlations, and phonon/magnon scattering.

2.2 The 3D Difference Pair Distribution Function

The three-dimensional difference pair distribution function (3D-ΔPDF) is defined as the Fourier transform of the diffuse scattering intensity:

$$\Delta\rho(\mathbf{r}) = FT[\, I_{diffuse}(\mathbf{Q}) \,] = FT[\, I_{total}(\mathbf{Q}) - I_{Bragg}(\mathbf{Q}) \,]$$

In real space, $\Delta\rho(\mathbf{r})$ is a three-dimensional map of interatomic correlation deviations. A positive value at vector $\mathbf{r}$ means there are *more* interatomic pairs separated by $\mathbf{r}$ than the average structure predicts; a negative value means *fewer*. Peaks in $\Delta\rho(\mathbf{r})$ at specific real-space positions directly identify the geometry of correlated displacements or ordered nano-domains.

The key practical relation connecting measurement to analysis is:

$$\Delta\rho(\mathbf{r}) \approx FT[\, I_{total}(\mathbf{Q}) - I_{Bragg}(\mathbf{Q}) \,] = FT[\, I_{cleaned}(\mathbf{Q}) \,]$$

where $I_{cleaned}(\mathbf{Q})$ is the diffuse volume after ring removal, Bragg punching, and backfill. The accuracy of $\Delta\rho$ therefore depends directly on the quality of each preprocessing stage.

> **Key Reference**
> Weber & Simonov, Z. Kristallogr. 227, 238–247 (2012). Simonov, Weber & Steurer, J. Appl. Cryst. 47, 2011–2018 (2014). These two papers establish the 3D-ΔPDF formalism and the punch-and-fill data-reduction strategy implemented in this package.

2.3 Coordinate Systems and Conventions

Two coordinate systems are used throughout the package:

- **Fractional HKL.** Reciprocal-lattice coordinates (h, k, l) measured in units of the reciprocal basis vectors **a***, **b***, **c***. Integer values correspond to Bragg positions; fractional values index the continuous diffuse volume.
- **Cartesian Q.** Physical momentum transfer in units of Å■¹, related to HKL by the UB matrix: **Q** = UB · [h, k, l]■. The magnitude $|\mathbf{Q}| = 2\pi/d$ (physics convention, as opposed to the crystallographic $1/d$ used in Mantid).

The **UB matrix** combines the orientation matrix U (aligning the crystal axes to the laboratory frame) and the reciprocal-metric matrix B (encoding the lattice parameters). Mantid files store the crystallographic UB (no 2π factor); neutron-diffuse reads this and multiplies by 2π to convert to the physics convention.

$$\mathbf{Q} = 2\pi \cdot \mathrm{UB}_{\mathrm{cryst}} \cdot [h, k, l] \quad (\text{Å}^{-1})$$

2.4 Powder Rings: Physical Origin

Polycrystalline material in the beam path — the sample cryostat, aluminium heat shields, sample holder, and sample capsule walls — produces *powder diffraction rings*: bands of elevated intensity at fixed $|\mathbf{Q}|$ values corresponding to the *d*-spacings of the polycrystalline material. For aluminium (FCC, a = 4.046 Å), the strongest ring is Al(111) at $|\mathbf{Q}|$ = 2.69 Å^{-1}.

Rings are not isotropic. Detector solid-angle coverage, absorption path length, and normalization artifacts modulate the ring intensity with azimuthal direction ϕ. A useful physical model for the ring contribution at a voxel with scattering vector magnitude $|\mathbf{Q}|$ and azimuthal angle ϕ is:

$$I_{\mathrm{ring}}(Q, \phi) = T(\phi) \times \Sigma_i A_i \cdot G(|Q| - q_i, \sigma_i)$$

where $G(|\mathbf{Q}|-q_i, \sigma_i)$ is a Gaussian radial profile for ring i, A_i is the per-ring amplitude, and $T(\phi)$ is the shared azimuthal texture function. The measured signal is:

$$I_{\mathrm{measured}}(Q, \phi) = I_{\mathrm{diffuse}}(Q) + I_{\mathrm{ring}}(Q, \phi)$$

Since the diffuse signal of interest does *not* share the ring's radial peak structure or azimuthal texture, it is in principle separable from the ring contribution.

3. Package Architecture

3.1 Overview

The package is installed from source under src/ndiff/ and follows a strict separation between the core data structure, I/O, the four algorithmic stages, an inpainting library, and visualization:

Module	Responsibility
ndiff/core.py	HKLVolume dataclass — the universal data carrier
ndiff/io/	Mantid NeXus reader, HDF5 load/save, ASCII HKL I/O
ndiff/preprocessing/	Powder-ring models, radial background, backfill
ndiff/analysis/	Bragg punch/backfill, 3D-ΔPDF FFT
ndiff/inpainting/	Symmetry fill, TV (Chambolle-Pock), RBF, biharmonic
ndiff/visualization/	Slice plots, radial profiles, interactive viewers
ndiff/utils/	UB matrix, d-spacing, Q↔HKL conversion utilities
examples/	Pipeline scripts and interactive viewer entry points

3.2 The HKLVolume Data Structure

Every stage of the pipeline operates on an HKLVolume dataclass defined in src/ndiff/core.py:

```
@dataclass
class HKLVolume:
data : NDArray[np.float64] # shape (nh, nk, nl) intensity
sigma : NDArray[np.float64] # shape (nh, nk, nl) standard deviation
mask : NDArray[np.bool_] # True = valid (not masked)
h_axis : NDArray # 1D fractional-HKL H coordinates
k_axis : NDArray # 1D fractional-HKL K coordinates
l_axis : NDArray # 1D fractional-HKL L coordinates
ub_matrix : NDArray[np.float64]# (3, 3) physics-convention UB (Angstrom^-1)
instrument: str # provenance label
```

Key methods provided by HKLVolume:

Method	Description
`hkl_grid()`	Returns (H, K, L) meshgrids of shape (nh, nk, nl)
`q_magnitude()`	Returns |Q| in Å■¹ for every voxel
`masked_data()`	Returns data with masked voxels set to NaN
`apply_mask(m)`	Updates the mask in place

3.3 File Formats

Input: Mantid NeXus

Raw input is a Mantid MDHistoWorkspace saved as a NeXus file (*_cc_sub_bkg.nxs). The reader (ndiff/io/mantid_nxs.py) extracts the signal, variance, mask, bin-edge arrays, and the UB matrix from the standard Mantid HDF5 hierarchy. The Mantid crystallographic UB is rescaled by 2π on read.

Native HDF5

All intermediate and output files are stored as HDF5 (.h5). Load with vol = ndiff.load(path); save with ndiff.save(vol, path). The ΔPDF output additionally stores the transform configuration and direct-lattice constants as HDF5 attributes for use by the interactive viewers.

4. Algorithms

4.1 Stage 1: Powder Ring Removal

4.1.1 Design Principle

Ring removal is strictly *subtractive*. The algorithm estimates only the azimuthally smooth ring intensity and subtracts it. It never masks or replaces voxels solely because they exhibit radial excess, because radial excess can be genuine anisotropic diffuse scattering.

4.1.2 Non-Parametric Patch-Based Method (Production)

The production algorithm is implemented in ndiff/preprocessing/radial_background.py as PatchedRadialRingModel. It processes the volume one H-slice (0kl plane) at a time and builds a non-parametric radial ring model in azimuthal patches:

1. Azimuthal patches. The reference plane (default hk0) defines the azimuthal angle ϕ = atan2(k_Q, h_Q). The full range $\phi \in [0, 2\pi)$ is divided into N overlapping patches, each weighted by a Hann window to ensure smooth blending across patch boundaries.

2. Robust radial profile. Within each azimuthal patch, voxels are binned by |Q| and a trimmed-mean profile is computed. The trimmed mean rejects high-tail outliers (Bragg peaks) and low-tail outliers (detector gaps), yielding a clean estimate of the combined diffuse + ring signal in each bin.

3. SNIP baseline estimation. A smooth diffuse baseline is estimated from the robust profile using a morphological opening: successive minimum and maximum filters with a kernel wider than the expected ring width. Light polynomial smoothing is applied. This is equivalent to the Sensitive Nonlinear Iterative Peak (SNIP) clipping algorithm common in nuclear spectroscopy.

4. Ring component. The ring component for each patch is: ring(|Q|) = max(0, prof – base). Taking the positive part ensures no diffuse signal is subtracted when the profile lies below the baseline (noise fluctuations).

5. Subtraction. Each voxel receives a Hann-weighted blend of the ring estimates from its two nearest azimuthal patches, interpolated to the voxel's exact |Q|. Cross-H confirmed ring shells and amplitude ceilings from adjacent H planes are propagated to prevent integer-H Bragg artifacts from masquerading as powder rings in nearby planes.

4.1.3 Factored Gaussian/SVD Model (Legacy)

The older algorithm (PatchedRingModel) remains in the package and is useful background. It fits a rank-1 factored model to the amplitude matrix:

$$\mathtt{A[i,\ P]} \approx \mathtt{A\_i} \times \mathtt{T[P]}$$

where index *i* runs over rings and *P* over azimuthal patches. A rank-1 SVD extracts per-ring amplitudes A_i and patch textures T[P]. A Fourier series $T(\phi)$ is then fitted to give a smooth, periodic texture function. The quality of the rank-1 approximation is monitored via rank1_variance; values ≥ 0.90 confirm the shared-texture assumption.

4.1.4 Diagnostic Metrics

Diagnostic	Interpretation
`rank1_variance`	Fraction of amplitude-matrix variance explained by rank-1 SVD. ≥ 0.90 confirms shared T(ϕ). Lower values suggest per-ring texture fitting is needed.
Per-patch counts	Sparse patches receive lower weight; uniform coverage is ideal.
Raw vs baseline profiles	Plot with plot_radial_profile() to visually verify ring isolation.

4.2 Stage 2: Bragg Peak Punching

Bragg punching is implemented in ndiff/analysis/bragg.py by the BraggRemover class. The recommended production mode is mode="both", which combines a lattice-aware integer-node pass followed by an hkl-agnostic search pass.

4.2.1 Integer-Node Path

The integer-node path exploits the known lattice geometry:

- **Enumerate.** All integer (h, k, l) nodes within the HKL volume extent are listed.
- **Detect.** A local HKL window of half-width detect_window_hkl is inspected around each node. The node is retained only if the local peak exceeds min_intensity and the local median by min_prominence.
- **Recentre.** The punch centre is moved to the measured local maximum within the detection window.
- **Fit shape (optional).** If integer_optimize_shape=True, the anisotropic covariance of the peak excess above a threshold fraction of the peak height is computed, yielding per-peak HKL ellipsoid radii. These are clipped by integer_fit_max_radius_hkl to prevent over-punching in sparse data.
- **Guard (H-slab).** The parameter integer_h_guard_hkl clips each integer-node punch to a slab of half-width H_{guard} centred on the integer-H plane. This prevents strong Bragg holes at H=0 from extending into the diffuse planes at H=±1/3 or H=±2/3.

4.2.2 Weak Bragg at Integer Nodes

Weak peaks at integer nodes can fall below the absolute min_intensity and min_prominence floors but still be sharp local outliers. The parameter integer_local_prominence_n_mad catches these by requiring the prominence to exceed a multiple of the local Median Absolute Deviation (MAD) within the detection window. Because this criterion is *locked to integer lattice nodes* (never applied at fractional positions), it cannot punch the q=1/3 diffuse planes. An additional small absolute floor integer_local_min_prominence avoids false positives in very flat regions.

4.2.3 Search Path (HKL-Agnostic)

The search path detects off-integer satellites and any sharp feature missed by the integer pass. At each |**Q**| shell, a robust background level is estimated as:

$$\texttt{bg\_threshold} = \texttt{median(I\_shell)} + \texttt{n\_mad} \times \texttt{MAD(I\_shell)}$$

Local maxima above this threshold and the absolute floor search_min_intensity are retained as punch centres. Because the search does not know the crystal lattice, structured diffuse planes must be explicitly protected:

- **Explicit exclusion.** A list of H-plane centres (search_exclude_h_centers) defines fixed exclusion slabs of half-width search_exclude_h_half_width.

• **Periodic exclusion (preferred).** search_exclude_h_fractions defines fractional parts modulo 1. For example, fractions [0.3333, 0.6667] protect every H = n ± 1/3 and n ± 2/3 plane across the full H range, covering higher-order satellites at H = ±4/3, ±5/3, … that a fixed-centre list would miss.

4.2.4 Punch Shape

Each identified peak is punched as an anisotropic ellipsoid in HKL space:

$$(h - h_0)^2/r_h^2 + (k - k_0)^2/r_k^2 + (l - l_0)^2/r_l^2 \leq 1$$

where (h_0, k_0, l_0) is the fitted punch centre and (r_h, r_k, r_l) are the HKL semi-axes. A guard margin is added to all radii. Optionally, the radii are scaled by the cube root of the peak intensity (relative to a reference), capped at max_radius_scale.

4.2.5 Direct Beam

The direct beam at **Q** = 0 is much broader than ordinary Bragg peaks and is treated separately. It is punched after all other peaks, using independent radii specified by incident_beam_ellipsoid_radii_hkl. The backfill of the direct-beam hole uses a specially shifted radial shell just outside the punch boundary, preventing the over-subtraction halo from contaminating the fill.

4.3 Stage 3: Bragg-Hole Backfill

After punching, masked voxels must be replaced with physically reasonable estimates before the Fourier transform. The dedicated wrapper is backfill_bragg() in ndiff/analysis/bragg_fill.py.

4.3.1 Q-Shell Fill (Recommended)

For ordinary Bragg holes, the robust diffuse level at the same |**Q**| provides the best estimate. For each |**Q**| bin (step size q_shell_step), the background is estimated as:

$$I_{fill} = \text{median}(I_{valid\_in\_bin}) + n_{mad} \times \text{MAD}(I_{valid\_in\_bin})$$

where the median and MAD are computed over all unmasked voxels in the |**Q**| shell. If a shell has fewer than q_shell_min_count valid voxels, the algorithm falls back to local shell interpolation. This method is physically motivated: the diffuse intensity at a given |**Q**| is relatively smooth and isotropic, so the shell level is a good estimate for all punched voxels at that |**Q**|.

4.3.2 Local Fill

For fast visual checks or sparse synthetic volumes, method="local" fills each connected punched component from a dilated-shell median of nearby valid voxels, falling back to the global median if too few neighbours are available.

4.3.3 General Inpainting Methods

For complex cases, the general inpainting pipeline in ndiff/inpainting/pipeline.py provides:

Method	Algorithm	Strengths	Limitations
Symmetry	Crystal Laue-symmetry equivalents (exact, no smoothing)	Exact; preserves all features	Fails when all equivalents are also masked
TV	Total-variation Chambolle-Pock primal-dual	Preserves sharp features; piecewise smooth	Slower for large masks; may over-smooth
RBF	scipy RBFInterpolator, thin-plate spline	Fast for small isolated masks	Intrinsic smoothing
Biharmonic	∇■u = 0 iterative relaxation	Very smooth fills	Slow for large masks

4.3.4 TV Inpainting: Mathematical Formulation

Total-variation inpainting solves the following constrained minimisation:

$$\min_u \frac{1}{2} \| W(u - f) \|^2 + \lambda \|\nabla u\|_1$$

where **f** is the observed data (arbitrary in the masked region), **W** is the diagonal mask operator (1 for valid, 0 for masked), $\nabla \mathbf{u}$ is the 3D forward finite-difference gradient, and λ is the regularisation parameter. This is solved by the Chambolle–Pock primal-dual algorithm with step sizes $\tau\sigma = 1/6$ and projection onto the ■∞ ball. TV preserves piecewise-smooth structures (sharp diffuse sheets and streaks) while suppressing noise in the filled region.

4.4 Stage 4: 3D-ΔPDF Transform

4.4.1 Correct FFT Recipe

The cleaned volume stores **Q**=0 at the *array centre* (index n//2), but NumPy's fftn expects the origin at index [0,0,0]. A correct, centred Fourier transform of the real, centrosymmetric I(**Q**) must therefore be:

$$\Delta\rho = \texttt{fftshift(fftn(ifftshift(I\_windowed))).real}$$

Applied step by step in ndiff/analysis/delta_pdf.py:

- **Step 1: Fill masked voxels.** Replace NaN (masked) voxels with 0. The backfilled volume should already be NaN-free; this is a safety guard.
- **Step 2: Optional Q-space crop.** Symmetrically crop to |H| ≤ h_max, |K| ≤ k_max, |L| ≤ l_max, keeping the array centred on Q=0.
- **Step 3: Subtract smooth background.** Compute I_bg = GaussianBlur(I, σ) with σ ≈ 1.5 r.l.u. and subtract: I_new = I – I_bg. This removes the broad separable diffuse envelope that would otherwise produce an axis cross in real space (see Section 4.4.3).
- **Step 4: Apodize.** Multiply by a separable window W(H,K,L) = w(H)×w(K)×w(L). The Hann window w(x) = cos²(πx/(2x_max)) (default) suppresses termination ripples from the finite |Q| range. Alternatives: Gaussian with width gaussian_sigma × Q_max, or no window.
- **Step 5: Remove DC.** Subtract the mean *after* windowing: I_w –= mean(I_w). This ensures Σ I = 0 exactly, zeroing the r=0 self-correlation spike. (Subtracting before windowing leaves a nonzero windowed sum, creating a spurious large peak at r=0.)
- **Step 6: Zero-pad symmetrically.** Pad to the next power-of-2 in each dimension, keeping Q=0 on the new centre. One-sided padding shifts the origin and breaks the ifftshift below. Zero-padding provides sinc-interpolation of the real-space grid; it does not increase intrinsic resolution, which is set by the Q_max and apodization window.
- **Step 7: Transform.** ifftshift → fftn → fftshift: move Q=0 to the corner, transform, then recentre r=0.
- **Step 8: Take real part.** The transform of centrosymmetric data I(Q)=I(–Q) is real; the imaginary part is numerical noise and a useful diagnostic.

4.4.2 Real-Space Axes

The real-space coordinate along axis *a* is computed from the FFT frequency grid and the UB matrix:

$$\Delta h = (h\_max - h\_min) / (n\_h - 1)$$

$$\texttt{freq\_h = fftshift(fftfreq(n\_h\_padded, d=}\Delta h\texttt{)) [cycles per HKL unit]}$$

$$\texttt{x\_a = freq\_h} \times 2\pi / |\texttt{UB[:,0]}| \text{ [Å]}$$

The factor 2π/|**a***| converts cycles per HKL unit to Å in direct space, where **a*** = UB[:,0] is the first column of the UB matrix (the reciprocal basis vector along H).

4.4.3 The Axis-Cross Artifact and Its Removal

A common artifact in 3D-ΔPDF maps is a bright cross along the y_K=0 and z_L=0 axes in the y_K–z_L plane. Diagnosis (confirmed on $TbTi_3Bi_4$ data, 2026-06-05) showed:

- The cross is present on planes with *no* Bragg peaks (e.g. H=1/3).
- The input has 0% masked voxels along the axis lines.
- Replacing the exact K=0 or L=0 input lines with neighbour averages has no effect.

The root cause is a broad, slowly-varying *diffuse envelope* centred near K=L=0. This envelope is approximately separable ($I \approx f(K) + g(L)$), and the Fourier transform of a separable function concentrates on the two principal axes. The Hann window, itself a centred separable hump, multiplies this effect. Subtraction of the scalar mean only removes the DC term; it leaves the envelope shape untouched.

The fix is smooth-background subtraction before windowing. Using a Gaussian blur with $\sigma \approx 1.5$ r.l.u. (per-axis control available):

$$\mathbf{I\_new = I - GaussianBlur(I, \sigma)}$$

This removes the separable envelope while preserving the oscillatory modulation (the genuine diffuse structure). As with all ΔPDF background subtractions, genuine very-long-period / low-*r* correlations at the same length scale as the background cannot be cleanly separated and are also attenuated.

> **Parameter recommendation**
> Use SUBTRACT_BG="0,1.5,1.5" to apply a per-axis Gaussian blur with σ_H=0 (slice-wise background, preserves H-layering), σ_K=1.5, σ_L=1.5 r.l.u. This is the validated default for the $TbTi_3Bi_4$ dataset.

4.4.4 Near-Origin Spike

A strong feature at r < ~3 Å remains in all 3D-ΔPDF maps. It originates from residual high-|Q| Bragg leakage, discontinuities at the punch-hole boundaries, and the direct-beam punch. Colour scales are therefore set from the p99 of |Δρ| at r > 3 Å so this near-origin spike does not dominate the display.

4.4.5 The Centring Bug (Fixed 2026-06-05)

Earlier code computed fftshift(fftn(data)) without ifftshift and used one-sided zero-padding. With Q=0 at the array centre, the missing ifftshift introduces a linear phase ramp $e^{-i\pi k} = (-1)^k$ across the output. Taking the real part then *flips the sign of real-space features by pixel parity*, so each correlation peak splits into mixed positive/negative lobes. This has been fixed in all current code; the regression test test_delta_pdf_centring_positive_peak guards against reintroduction.

5. Installation and Setup

5.1 Requirements

Package	Version	Purpose
Python	≥ 3.10	Core language
numpy	≥ 1.24	Array operations, FFT
scipy	≥ 1.10	Interpolation, Gaussian filters
h5py	≥ 3.8	HDF5 file I/O
matplotlib	≥ 3.7	Visualization

5.2 Install from Source

```
git clone https://github.com/user/neutron-diffuse
cd neutron-diffuse
pip install -e ".[dev]"
```

The [dev] extra installs pytest, ruff, and mypy for the test suite and linters.

5.3 Recommended Runtime Environment

The recommended environment uses the sci-general conda environment with all dependencies pre-installed. Set the following shell variables before running any script:

```
export PY=/opt/homebrew/Caskroom/miniforge/base/envs/sci-general/bin/python
export PYTHONPATH=src
export MPLCONFIGDIR=/private/tmp/ndiff-mpl
```

MPLCONFIGDIR keeps Matplotlib cache files outside the repository. If your Python 3.10+ environment already has the dependencies active, replace $PY with python3.

5.4 Verifying the Installation

```
PYTHONPATH=src python -c "import ndiff; print(ndiff.__version__)"
PYTHONPATH=src python -m pytest -o addopts='' tests/
```

6. End-to-End Workflow

6.1 One-Command Pipeline

The script examples/run_pipeline.py orchestrates all four stages automatically. It detects which output files already exist and skips those stages, enabling fast incremental reprocessing:

```
PYTHONPATH=src MPLCONFIGDIR=/tmp/mpl \
python examples/run_pipeline.py
```

Key environment overrides:

Variable	Effect
DATA_FILE	Path to input .nxs file (default: auto-detected 22 K file)
NO_VIEWER=1	Skip GUI stages; stop after writing _delta_pdf.h5
FORCE=1	Recompute every stage even if output exists
FORCE_FROM=rings|punch|backfill|pdf	Recompute from the named stage onward
SLICE_AXIS=H|K|L	Direction along which ring removal iterates planes (default H)

6.2 Stage-by-Stage Commands

Stage 1: Ring Removal

```
PYTHONPATH=src MPLCONFIGDIR=/tmp/mpl RING_PRESET=cc_on \
python examples/remove_rings_3d.py
```

Output: data/processed/*_ringremoved.h5

Use RING_PRESET=cc_off for more aggressive subtraction on noisy data.

Stage 2: Bragg Punch

```
PYTHONPATH=src MPLCONFIGDIR=/tmp/mpl \
PUNCH_PRESET=cc_on MODE=both MIN_I=0.8 MIN_PROM=0.8 \
INTEGER_FIT_POSITION=1 INTEGER_FIT_SHAPE=1 \
INTEGER_H_GUARD=0.12 \
SEARCH_EXCLUDE_H_FRACTIONS=0.3333,0.6667 \
SEARCH_EXCLUDE_H_WIDTH=0.08 \
python examples/punch_bragg_3d.py
```

Output: data/processed/*_braggpunched.h5

Stage 3: Backfill

```
PYTHONPATH=src METHOD=q_shell \
python examples/backfill_bragg_3d.py
```

Output: data/processed/*_braggpunched_backfilled.h5

Stage 4: 3D-ΔPDF

```
PYTHONPATH=src MPLCONFIGDIR=/tmp/mpl \
SUBTRACT_BG="0,1.5,1.5" CROP_H=4 CROP_K=8 CROP_L=15 \
APODIZE=gaussian GAUSSIAN_SIGMA=0.4 \
python examples/delta_pdf.py
```

Outputs: examples/_delta_pdf.h5 and PNG central-cut images.

6.3 Multi-Temperature Workflow ($TbTi_3Bi_4$)

The reference dataset consists of three temperatures. Run the pipeline for each temperature by overriding DATA_FILE:

```
# 22 K
NO_VIEWER=1 \
DATA_FILE="data/raw/TbTi3Bi4_22K_mmm_..._cc_sub_bkg.nxs" \
python examples/run_pipeline.py

# 45 K
NO_VIEWER=1 \
DATA_FILE="data/raw/TbTi3Bi4_45K_..._cc_sub_bkg.nxs" \
python examples/run_pipeline.py

# 100 K
NO_VIEWER=1 \
DATA_FILE="data/raw/TbTi3Bi4_100K_..._cc_sub_bkg.nxs" \
python examples/run_pipeline.py
```

To generate persistent per-temperature ΔPDF files in data/processed/ use OUT_FILE and PROC_FILE:

```
PROC_FILE="data/processed/TbTi3Bi4_22K_..._backfilled.h5" \
OUT_FILE="data/processed/TbTi3Bi4_22K_mmm_delta_pdf.h5" \
SUBTRACT_BG="0,1.5,1.5" CROP_H=4 CROP_K=8 CROP_L=15 \
APODIZE=gaussian GAUSSIAN_SIGMA=0.4 \
python examples/delta_pdf.py
```

6.4 Python API

All pipeline stages can be called programmatically:

```
import ndiff
from ndiff.analysis import BraggRemover, backfill_bragg, compute_delta_pdf

# Load ring-removed volume
vol = ndiff.load("data/processed/sample_ringremoved.h5")

# Bragg punch
remover = BraggRemover(
mode="both",
punch_radii=(0.09, 0.12, 0.45),
min_intensity=0.8,
min_prominence=0.8,
integer_optimize_position=True,
integer_optimize_shape=True,
integer_h_guard_hkl=0.12,
integer_local_prominence_n_mad=8.0,
search_n_mad=4.0,
search_exclude_h_fractions=(1/3, 2/3),
search_exclude_h_half_width=0.08,
incident_beam_ellipsoid_radii_hkl=(0.15, 0.50, 1.00),
)
punched = remover.apply(vol)

# Backfill
filled = backfill_bragg(punched, method="q_shell")

# 3D-DeltaPDF
dpdf = compute_delta_pdf(
filled,
apodization="gaussian",
gaussian_sigma=0.4,
crop_hkl=(4, 8, 15),
subtract_smooth_bg=(0, 1.5, 1.5),
)
print(f"DeltaPDF shape: {dpdf.data.shape}")
print(f"Real-space range: x +/-{dpdf.x_axis.max():.1f} A")
```

7. Configuration Reference

7.1 Ring Removal Parameters

Parameter	Type	Default	Description
RING_PRESET	str	cc_on	Preset: cc_on (conservative) or cc_off (aggressive)
Q_STEP	float	0.02	Radial bin width for profile computation (Å■¹)
N_FOURIER	int	8	Number of Fourier terms for T(ϕ) fit
PROFILE_METHOD	str	trimmed_mean	Bin statistic: trimmed_mean or median
TEXTURE_Q_SMOOTH	float	0.1	Smoothing σ for texture along |Q| (Å■¹)
SLICE_AXIS	str	H	Axis along which to iterate slices: H, K, or L

7.2 Bragg Punch Parameters

Parameter	Type	Default	Description
MODE	str	both	Detection: integer, search (auto), or both
PUNCH_PRESET	str	cc_on	Preset for punch radii and thresholds
MIN_I	float	0.8	Minimum intensity threshold for Bragg detection
MIN_PROM	float	0.8	Minimum local-median prominence
INTEGER_FIT_POSITION	0/1	1	Optimize punch centre to measured maximum
INTEGER_FIT_SHAPE	0/1	1	Fit anisotropic punch radii from data
INTEGER_H_GUARD	float	0.12	H-slab half-width for integer-node punches
SEARCH_EXCLUDE_H_FRACTIONS	str	0.3333,0.6667	Periodic H exclusions (fractional parts mod 1)
SEARCH_EXCLUDE_H_WIDTH	float	0.08	Half-width of search-exclusion slabs
INCIDENT_ELLIPSOID_R_HKL	str	0.15,0.50,1.00	Direct-beam punch ellipsoid semi-axes (HKL)

7.3 Backfill Parameters

Parameter	Type	Default	Description
METHOD	str	q_shell	Fill method: q_shell, local, tv, symmetry, symmetry+tv
Q_SHELL_STEP	float	0.05	Bin width for q-shell fill (Å■¹)
Q_SHELL_MIN_COUNT	int	10	Min valid voxels per bin; fall back to local if fewer
TV_LAM	float	0.1	TV regularisation λ (higher = smoother fill)
TV_ITER	int	300	Chambolle-Pock iterations

7.4 3D-ΔPDF Transform Parameters

Parameter	Type	Default	Description
APODIZE	str	gaussian	Window function: hann, gaussian, or none
GAUSSIAN_SIGMA	float	0.4	Gaussian window width as fraction of Q_max
CROP_H / CROP_K / CROP_L	float	4 / 8 / 15	Symmetric Q-crop limits in HKL
SUBTRACT_BG	str	0,1.5,1.5	Per-axis Gaussian blur σ for background subtraction (H,K,L r.l.u.)
REAL_SPACE_ANGSTROM	0/1	1	Output real-space axes in Å (1) or HKL units (0)
OUT_FILE	str	examples/_delta_pdf.h5	Output HDF5 path for the ΔPDF volume

8. Visualization and Interactive Exploration

8.1 Visualization API Overview

All visualization functions live in ndiff.visualization and follow a primitive-first design: each function accepts an HKLVolume, draws into a caller-supplied Matplotlib Axes or Figure, and returns it. This makes the same calls work in one-shot scripts, IPython sessions, Jupyter notebooks, and the interactive viewer scripts.

```
from ndiff.visualization import (
extract_slice, plot_slice,
plot_radial_profile, plot_azimuthal_map,
plot_overview, SliceData,
)
```

8.2 plot_slice()

Two-dimensional intensity slice through an HKLVolume. The plane argument is read as (horizontal, vertical) and selects the two displayed axes; the remaining axis is cut at value:

Plane	x-axis	y-axis	Fixed (cut by value)
<tt>'kl'</tt> or <tt>'0kl'</tt>	K	L	H
<tt>'hl'</tt> or <tt>'h0l'</tt>	H	L	K
<tt>'hk'</tt> or <tt>'hk0'</tt>	H	K	L

```
# Log-scale background ring view
plot_slice(bkg, "kl", value=0.0, log_scale=True)

# Exact fractional plane with manual colour limits
plot_slice(data, "hk", value=0.3333, interp=True, vmin=0.0, vmax=0.4)
```

Off-grid interpolation. Pass interp=True to linearly interpolate between the two bracketing planes so the exact value is honoured. The interpolation is NaN-aware: where only one bracketing plane is valid, that value is used.

8.3 plot_radial_profile() and plot_azimuthal_map()

```
# Radial profile with Al(111) ring marker
plot_radial_profile(data, mark_q=[2.69])

# Azimuthal texture of the Al(111) ring
plot_azimuthal_map(data, q_center=2.69)
```

plot_radial_profile bins voxels by |**Q**| and plots the mean or median intensity. plot_azimuthal_map bins by azimuthal angle ϕ within a thin |**Q**| shell, showing the ring texture T(ϕ).

8.4 plot_overview()

```
fig = plot_overview(data, log_scale=True)
```

A 2×2 diagnostic figure: KL (H=0), HL (K=0), HK (L=0) slices and a radial profile. The first look at any new volume.

8.5 Interactive Viewers

Cleanup QA Viewer

Four-panel view (raw → ring-removed → Bragg-punched → backfilled) with H/K/L plane selector and cut-position slider. Use to verify that integer-H Bragg peaks are cleanly removed and fractional-H diffuse planes are preserved:

```
PYTHONPATH=src MPLCONFIGDIR=/tmp/mpl \
PUNCH_PRESET=cc_on MODE=both MIN_I=0.8 MIN_PROM=0.8 \
INTEGER_FIT_POSITION=1 INTEGER_FIT_SHAPE=1 INTEGER_H_GUARD=0.12 \
SEARCH_EXCLUDE_H_FRACTIONS=0.3333,0.6667 SEARCH_EXCLUDE_H_WIDTH=0.08 \
H_VALUE=0.3333 \
python examples/explore_slice.py
```

3D-ΔPDF Orthoslice Viewer (Recommended)

All three orthogonal real-space planes simultaneously (x_H–y_K, x_H–z_L, y_K–z_L) with independent cut sliders, a contrast multiplier, and a unit-cell gridline toggle:

```
PYTHONPATH=src MPLCONFIGDIR=/tmp/mpl \
PDF_FILE=data/processed/TbTi3Bi4_22K_mmm_delta_pdf.h5 RMAX=50 \
python examples/explore_delta_pdf_ortho.py
```

Multi-Temperature Comparison Grid

A 3×3 grid (rows = 22 K/45 K/100 K, columns = three orthogonal cuts) with shared cut sliders and a single global colour scale so intensities are directly comparable across temperatures:

```
PYTHONPATH=src MPLCONFIGDIR=/tmp/mpl \
PDF_22K=data/processed/TbTi3Bi4_22K_mmm_delta_pdf.h5 \
PDF_45K=data/processed/TbTi3Bi4_45K_delta_pdf.h5 \
PDF_100K=data/processed/TbTi3Bi4_100K_delta_pdf.h5 \
python examples/explore_delta_pdf_multi.py
```

9. Worked Examples

9.1 Quick Inspection of a New Dataset

Load a raw volume and display the 2×2 diagnostic overview:

```
import ndiff
from ndiff.visualization import plot_overview

vol = ndiff.load("path/to/sample.nxs") # or .h5
fig = plot_overview(vol, log_scale=True)
fig.savefig("overview.png", dpi=120)
```

9.2 Locating and Profiling a Powder Ring

```
from ndiff.visualization import plot_radial_profile, plot_azimuthal_map
import matplotlib.pyplot as plt

# Step 1: identify ring positions from radial profile
fig, ax = plt.subplots()
plot_radial_profile(vol, ax=ax, mark_q=[2.69, 4.39, 5.07])
plt.title("Radial profile: Al ring positions")

# Step 2: check azimuthal texture T(phi) at Al(111)
plot_azimuthal_map(vol, q_center=2.69, q_width=0.05)
plt.title("Al(111) ring texture")
```

9.3 Checking Bragg Punch Quality

Compare an integer-H plane (Bragg present) to a fractional-H diffuse plane:

```
import ndiff
from ndiff.visualization import plot_slice
import matplotlib.pyplot as plt

punched = ndiff.load("data/processed/..._braggpunched.h5")

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Integer-H: check all Bragg holes are punched
plot_slice(punched, "kl", value=0.0, ax=axes[0],
log_scale=True, title="H=0 (integer): Bragg punched")

# Fractional-H: check diffuse is preserved
plot_slice(punched, "kl", value=0.3333, interp=True, ax=axes[1],
log_scale=True, title="H=1/3 (fractional): diffuse intact")

fig.tight_layout()
fig.savefig("bragg_check.png", dpi=110)
```

9.4 Computing and Displaying the 3D-ΔPDF

```
import ndiff
from ndiff.analysis import compute_delta_pdf
import matplotlib.pyplot as plt

filled = ndiff.load("data/processed/..._backfilled.h5")

dpdf = compute_delta_pdf(
filled,
apodization="gaussian",
gaussian_sigma=0.4,
crop_hkl=(4, 8, 15),
subtract_smooth_bg=(0, 1.5, 1.5),
)
ndiff.save(dpdf, "output_delta_pdf.h5")

# Plot the central hk0 slice
hk0 = dpdf.slice_hk0() # h-k plane at l=0
vmax = abs(hk0.data).max()

fig, ax = plt.subplots(figsize=(7, 6))
im = ax.imshow(hk0.data.T, origin="lower",
extent=[hk0.x_axis[0], hk0.x_axis[-1],
hk0.y_axis[0], hk0.y_axis[-1]],
cmap="RdBu_r", vmin=-vmax, vmax=vmax)
ax.set_xlabel(r"$x_H$ (Å)")
ax.set_ylabel(r"$y_K$ (Å)")
fig.colorbar(im, ax=ax, label="Δρ (arb.)")
fig.savefig("delta_pdf_hk0.png", dpi=150)
```

10. Testing

10.1 Running the Test Suite

```
# Full suite with coverage
PYTHONPATH=src python -m pytest -o addopts='' tests/

# Lint (ruff)
python -m ruff check src/ tests/

# Type check (mypy)
python -m mypy src/ndiff --ignore-missing-imports
```

Note: The -o addopts='' flag is required because the project uses conda rather than a virtualenv; without it pytest may pick up unexpected plugins.

10.2 Key Regression Tests

Test	What it verifies
`test_delta_pdf_centring_positive_peak`	Guards the FFT centring fix: verifies a positive cosine input produces a positive real-space peak (not a sign-flipped lobe).
Ring subtraction (synthetic)	Verifies that a known Gaussian ring on a flat background is cleanly subtracted without affecting the flat region.
Bragg punch / search modes	Checks that integer-node and search-mode masks are consistent with expected punch radii and guard conditions.
Symmetry fill	Verifies that inverse-variance-weighted symmetry averaging correctly fills masked voxels using Laue equivalents.
TV inpainting convergence	Checks that the Chambolle-Pock primal-dual solver converges within the specified iteration budget.

11. Known Limitations and Troubleshooting

11.1 Near-Origin Spike

Symptom. A very strong feature at r < ~3 Å in all ΔPDF maps that dominates the colour scale.

Cause. Residual high-|Q| Bragg leakage, discontinuities at punch-hole boundaries, and the direct-beam punch.

Mitigation. Set colour scales from the p99 of |Δρ| at r > 3 Å (this is the default in all viewer scripts). For quantitative analysis of short-range correlations, tapered punch boundaries and softer high-|Q| windows are planned for a future release.

11.2 Axis Cross in ΔPDF

Symptom. A bright cross along y_K=0 and z_L=0 axes in the real-space map.

Cause. Residual separable diffuse envelope (see Section 4.4.3).

Fix. Use SUBTRACT_BG="0,1.5,1.5" (or the Python API parameter subtract_smooth_bg=(0, 1.5, 1.5)).

11.3 Residual Ring After Subtraction

Symptom. Faint ring pattern still visible after ring removal.

Causes and mitigations.

- Gaussian width σ_i too narrow: widen via Q_STEP and re-run detection.
- Texture mismatch (rank1_variance < 0.90): check diagnostic and consider per-ring T_i(ϕ) fitting (planned feature).
- Ring preset mismatch: try RING_PRESET=cc_off for noisier or more strongly ringed data.

11.4 Search Mode Punching Diffuse Structure

Symptom. Structured diffuse on fractional-H planes is partially masked.

Fix. Add the fractional H values to the periodic exclusion list: SEARCH_EXCLUDE_H_FRACTIONS=0.3333,0.6667. Use the periodic form rather than an explicit centre list to catch higher-order satellites automatically.

11.5 Rank-1 SVD Failure in Ring Model

Symptom. rank1_variance < 0.90 after ring fitting.

Meaning. Different rings have significantly different azimuthal textures; the shared T(ϕ) model is insufficient.

Current workaround. Use the non-parametric production path (PatchedRadialRingModel), which handles per-patch textures without assuming a shared T(ϕ). Per-ring T_i(ϕ) fitting is a planned improvement for the factored model.

11.6 Performance Notes

Benchmarks are for a 401×401×301 volume (~48 M voxels) on a typical laptop. Ring removal is the bottleneck; it is embarrassingly parallel per H-slice and a future release will add optional concurrent.futures.ProcessPoolExecutor parallelisation.

Operation	Typical time	Notes
Ring removal (per-slice)	~2–5 min	Sequential per-H-slice; embarrassingly parallel
Bragg punch (detection + mask)	~1–2 min	Scales with n_peaks
Backfill (q-shell)	~30 sec	Linear in n_voxels
3D-ΔPDF (FFT + windowing)	~10 sec	Dominated by FFT of zero-padded array

12. Glossary

Apodization. A window function applied to I(Q) before FFT to suppress termination ripples from the finite Q range. Common choices: Hann, Gaussian.

Backfill. Replacement of masked (punched) voxels with estimated background intensities derived from surrounding valid voxels.

Biharmonic. A fill based on iterative solution of $\nabla^2 u=0$; produces very smooth interpolations.

Bragg peaks. Sharp peaks at integer (and satellite) reciprocal-lattice nodes; encode the average periodic structure.

Centrosymmetric. Having inversion symmetry $I(Q)=I(-Q)$; ensures the Fourier transform is real-valued.

Diffuse scattering. Broad, continuous intensity encoding structural disorder, short-range order, and dynamic correlations.

Fractional HKL. Reciprocal-lattice coordinates (h,k,l) in units of a*, b*, c*; integers are Bragg positions.

Hann window. A cosine-squared taper $w(x)=\cos^2(\pi x/2x\_max)$; smoothly suppresses the signal to zero at the Q boundary.

HKLVolume. The central data structure: a 3D array plus axes, mask, σ, UB matrix, and instrument label.

Laue class. Crystal point-group symmetry class (e.g. mmm, m3m, 4/mmm) determining equivalent Q positions.

MAD. Median Absolute Deviation; a robust measure of spread: MAD = median(|x – median(x)|).

Powder ring. Azimuthally smooth band of intensity at fixed |Q| from polycrystalline material in the beam.

Punch. Mask and remove a Bragg peak by setting a region of voxels invalid.

Q. Cartesian momentum transfer vector in $Å^{-1}$; $Q = UB\cdot[h,k,l]^T$; $|Q| = 2\pi/d$ (physics convention).

SNIP. Sensitive Nonlinear Iterative Peak clipping; a morphological method for baseline estimation in 1D profiles.

Total Variation (TV). Regulariser $||\nabla u||_1$ promoting piecewise-smooth solutions; preserves sharp diffuse sheets.

UB matrix. Orientation (U) × metric (B) matrix; $Q = UB\cdot[h,k,l]^T$.

3D-ΔPDF. Three-dimensional difference pair distribution function; the Fourier transform of diffuse scattering, mapping to real-space pair correlations.

13. References

[1] T. Weber and A. Simonov, “The three-dimensional pair distribution function analysis of disordered single crystals: basic concepts,” *Z. Kristallogr.* **227**, 238–247 (2012). DOI: 10.1524/zkri.2012.1504. **Establishes the 3D-ΔPDF formalism.**

[2] A. Simonov, T. Weber, and W. Steurer, “Diffuse scattering from the disordered intermetallic compound $TbFe_2(Si_{1-x}Al_x)_4$ ($0 \leq x \leq 0.67$),” *J. Appl. Cryst.* **47**, 2011–2018 (2014). DOI: 10.1107/S1600576714023668. **3D-ΔPDF punch-and-fill strategy.**

[3] A. Chambolle and T. Pock, “A First-Order Primal-Dual Algorithm for Convex Problems with Applications to Imaging,” *J. Math. Imaging Vision* **40**, 120–145 (2011). DOI: 10.1007/s10851-010-0251-1. **Primal-dual TV inpainting algorithm used in ndiff.**

[4] M. Bertalmio, G. Sapiro, V. Caselles, and C. Ballester, “Image inpainting,” *Proc. ACM SIGGRAPH*, 417–424 (2000). DOI: 10.1145/344779.344972. **Origin of PDE-based diffusion inpainting.**

[5] M. Bertero and P. Boccacci, *Introduction to Inverse Problems in Imaging*, IOP Publishing, Bristol, UK (1998). ISBN: 978-0750304351. **General theory of regularised inverse problems.**

[6] Mantid Project, “Mantid — Manipulation and Analysis Toolkit for Instrument Data,” *Mantid* (2013–present). www.mantidproject.org. **Data reduction and MDHistoWorkspace I/O.**

[7] M. Ryan and D. Giannakis, “Sensitive nonlinear iterative peak-clipping algorithm (SNIP) for background estimation in spectrometry,” *Nucl. Instrum. Methods Phys. Res. B* **34**, 396–402 (1988). **SNIP baseline algorithm used in radial profile processing.**